

```
//Add the vectors and store result in vector C
void vector_add(int *a,int *b, int *c)
{
    int i=0;
    for (i=1;i<=N;i++)
    {
        c[i]=a[i]+b[i];
    }
}

int main()

{

    int a[N], b[N], c[N];
    int i=0;

    //Initialize the vectors with values from 1 to 100
    and its double in another array
    {
        a[i]=i;
        b[i]=2*i;
    }

    //Call function vector_add to display the result
    vector_add(a,b,c);

    //Print the resultant array.
    for (i=1;i<=N;i++)
    {
        printf("%d %d %d\n", a[i],b[i],c[i] );
    }

    system("pause");
    return 0;

}
```

This very simple serial vector addition program creates two arrays of integer values, and adds them using the `vector_add` function.

Compile the code using the following command:

```
gcc sequential_vector.c -o sequential
```

Run it using the command given below:

```
./sequential
```

In the above program, the processor runs each task sequentially, one after the other. Looping in the above program works sequentially as it starts with the first index and computes consequentially till the last index, and then

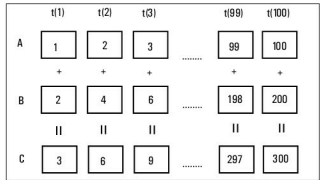


Figure 1: Sequential vector addition

exits the program. It is a single-threaded execution. The figure above would make the serial execution concept clear, where $t(x)$ represents the time slot of execution.

Now, CUDA gives us the functionality to perform the same operation in parallel. What it does essentially is offload the data parallel sections to the GPU device and send the result back after computation. In what follows, you will get an insight into launching kernels, writing a device and host code, and performing the same serial vector addition program given above, in parallel.

Here is a simple vector addition code in CUDA.

```
#include <cuda.h>
#include<stdio.h>
#define N 100
#define numThread 1 // in this example we keep one thread in
one block
#define numBlock 100 // in this example we use 100 blocks

__global__ void vector_add( int *a, int *b, int *c ) {

    // keep track of the index
    int tid = blockIdx.x;

    while ( tid < N ) {
        c[tid] = a[tid] + b[tid];
        tid = tid+ numBlock; // shift by the total number of
blocks, i.e. 100 in our case
    }
}

int main( void ) {
    int *a, *b, *c;
    int *dev_a, *dev_b, *dev_c;

    // allocate the memory on the CPU
    a = (int*)malloc( N * sizeof(int) );
    b = (int*)malloc( N * sizeof(int) );
    c = (int*)malloc( N * sizeof(int) );
```

```
//Initialize the vectors with values from 1 to 100 and its
double in another array
for (int i=0; i<N; i++) {
```